04 Latent Space

BY NILUSHANAN KULASINGHAM

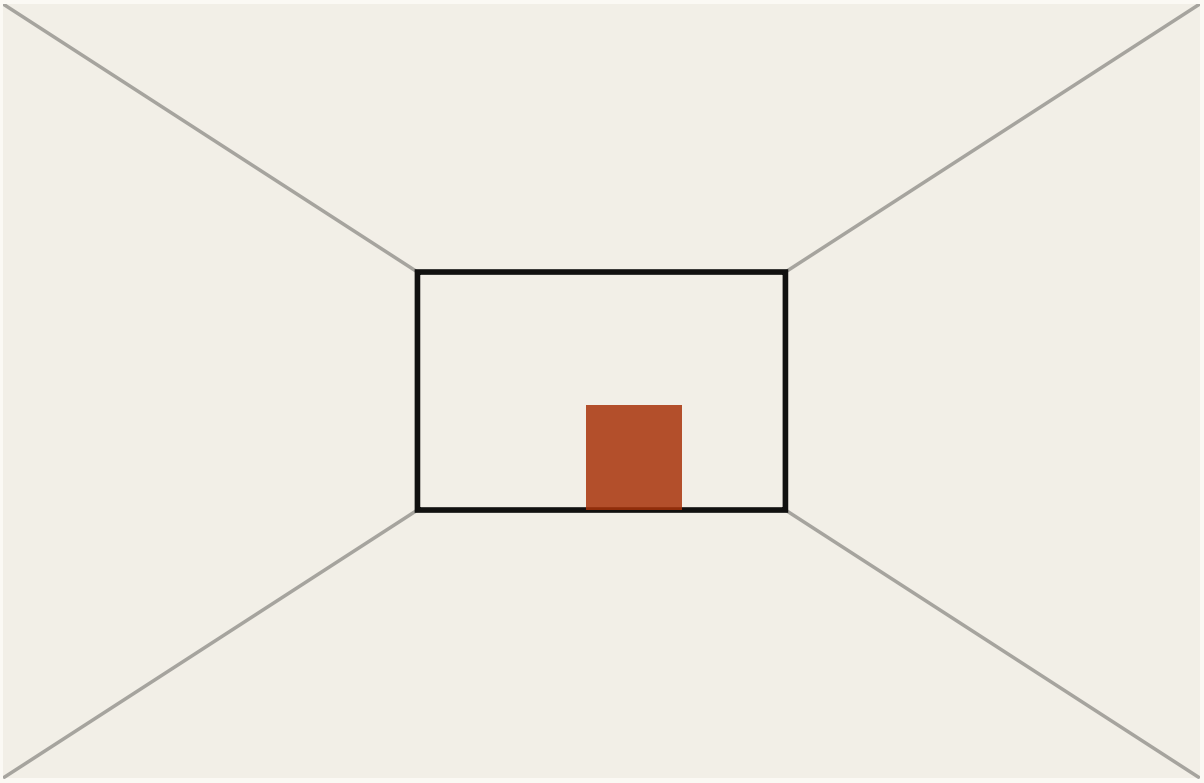
11 MIN READ · INTERACTIVE

> A camera measures tens of thousands of numbers. The decision needs two or three. What happens at the squeeze in between, and why that narrow point sets the ceiling on everything after it.

Stand in a doorway and look into a room. Your eyes are taking in something like a hundred million measurements a second.

Now decide whether to walk forward.

You needed almost none of it. How far is the far wall. Is there a way through, and roughly where. That is two or three numbers, and everything else your eyes collected was, for this decision, decoration.



THE ROOM THOSE TWO NUMBERS DECODE TO

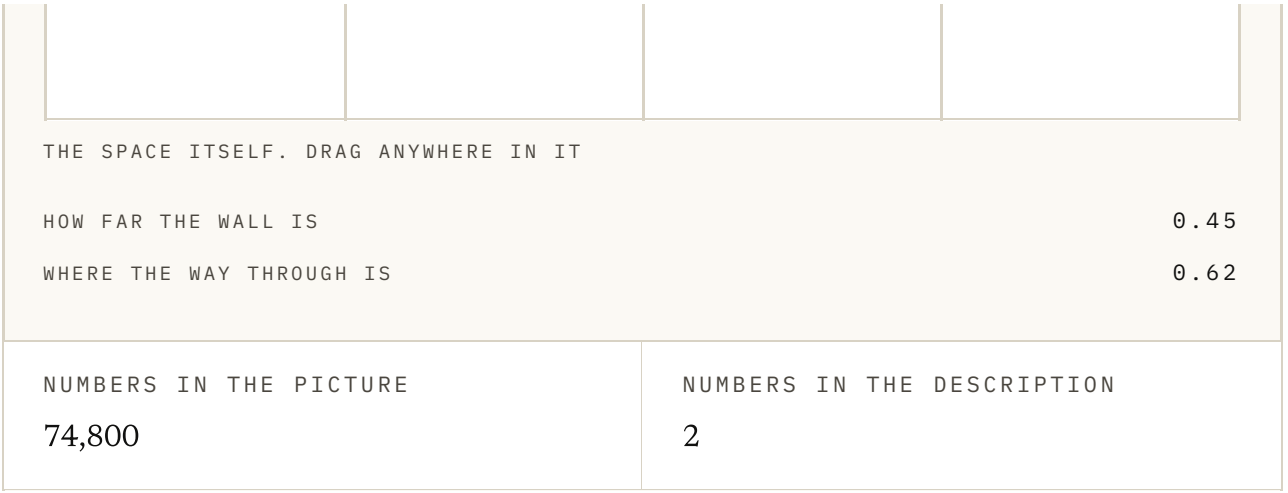


FIG. 4.1 Drag anywhere in the square on the right. Every point in it decodes to a room, and the room is computed from those two numbers rather than looked up. The picture is tens of thousands of numbers. The thing that produced it is two.

Those two or three numbers have a name. They are the *latent* (latent means hidden or not directly visible: the numbers are never shown to you, and nothing labels them, but the picture is what it is because of them) description of the scene, and the gap between those two counts is what this chapter is about. Somewhere between the camera and the decision there has to be a squeeze. What comes out the other side is all the rest of the system ever gets.

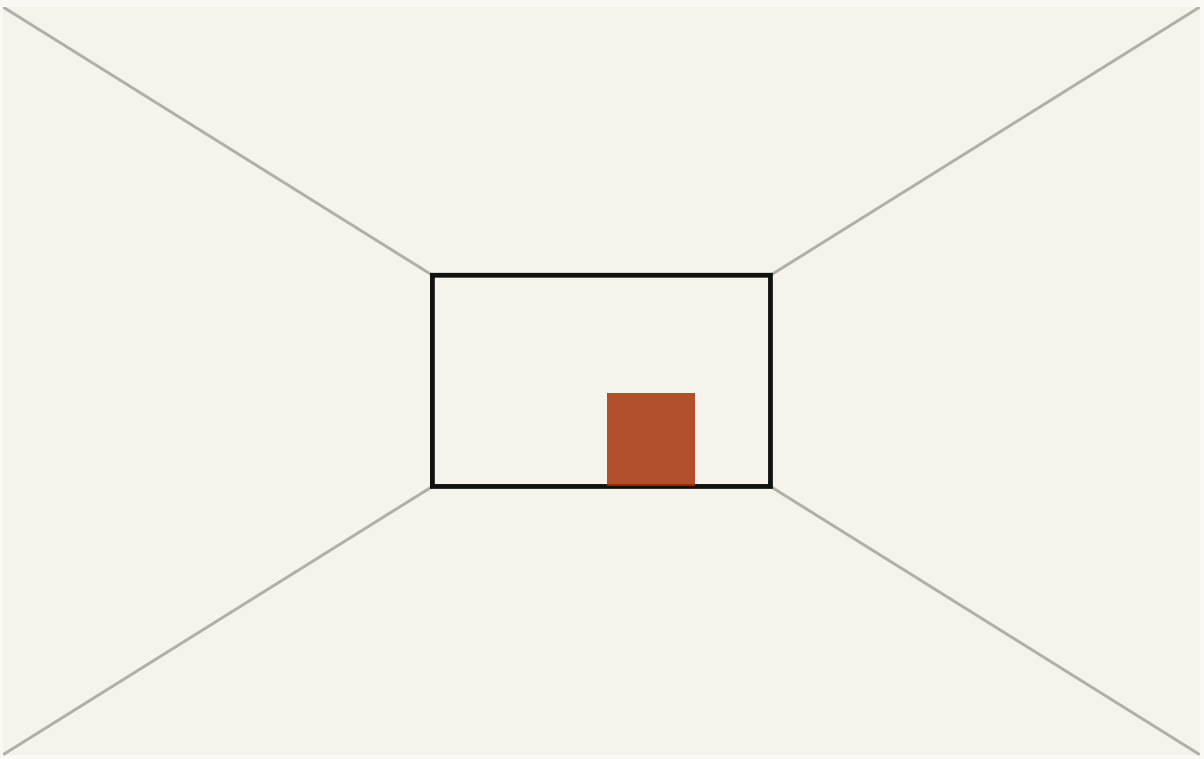
The squeeze

Put a narrow opening in the middle of a system and force everything through it.

That is the whole architecture. On one side, something that takes the picture and produces a short list of numbers. On the other, something that takes the short list and tries to rebuild the picture. Train the pair together, and the only way to succeed is for the short list to carry what the picture was actually made of.

> The two halves have names that are worth knowing, because every paper uses them. The first is the *encoder*, the second the *decoder*, and the narrow bit in the middle is the *bottleneck*.

Nobody has to decide what the short list should contain. That falls out. If five numbers are all you get and the pictures were generated by five things changing, the cheapest way to rebuild them is to recover those five things.



HOW FAR THE WALL IS
WHERE THE WAY THROUGH IS
WHICH WAY THE CAMERA IS TIPPED
GRAVEL ON THE FLOOR
SENSOR NOISE
HOW BRIGHT IT IS

COULD YOU DECIDE WHAT TO DO?

Yes. Two numbers is the whole decision.

WHAT THE EXTRA WIDTH BOUGHT

Nothing wasted yet. Everything through the squeeze changes what you would do.

FIG. 4.2 The scene here is generated from six numbers, and the squeeze keeps the first few. Slide it up from zero. Two numbers in and the decision is already settled; everything after that is gravel and sensor noise, which are real, and which change nothing about what you would do.

Watch which numbers arrive first in that figure, because it is doing the work of the whole chapter. The useful thing about a bottleneck is not that it is narrow. It is that being narrow forces an ordering, and the ordering it forces is roughly what matters

first.

Roughly. A squeeze on its own has no idea what you plan to do with the result. It will happily spend its width on whatever is hardest to rebuild, which is not the same as whatever you needed.

This is not a thought experiment. Ha and Schmidhuber's world model put a squeeze on the front of exactly this shape: every video frame from a driving game crushed down to thirty-two numbers, with everything after that working only from those. PlaNet does the same job from raw pixels and then plans in what comes out, never rebuilding a frame in order to decide anything.

Thirty-two numbers, for a picture that arrived as tens of thousands. That ratio is the point, and it is roughly the ratio between what a camera measures and what a decision turns on.

A place, not a list

Here is the shift that takes a while to land.

Those numbers are not a summary of the picture. They are coordinates. They name a point in a space, and the space has neighbours, directions and distances, none of which the picture had.

That means you can do things there that make no sense in pixels. You can ask what is nearby. You can move in a direction and watch the world change smoothly in some particular way. You can take two rooms and stand between them.

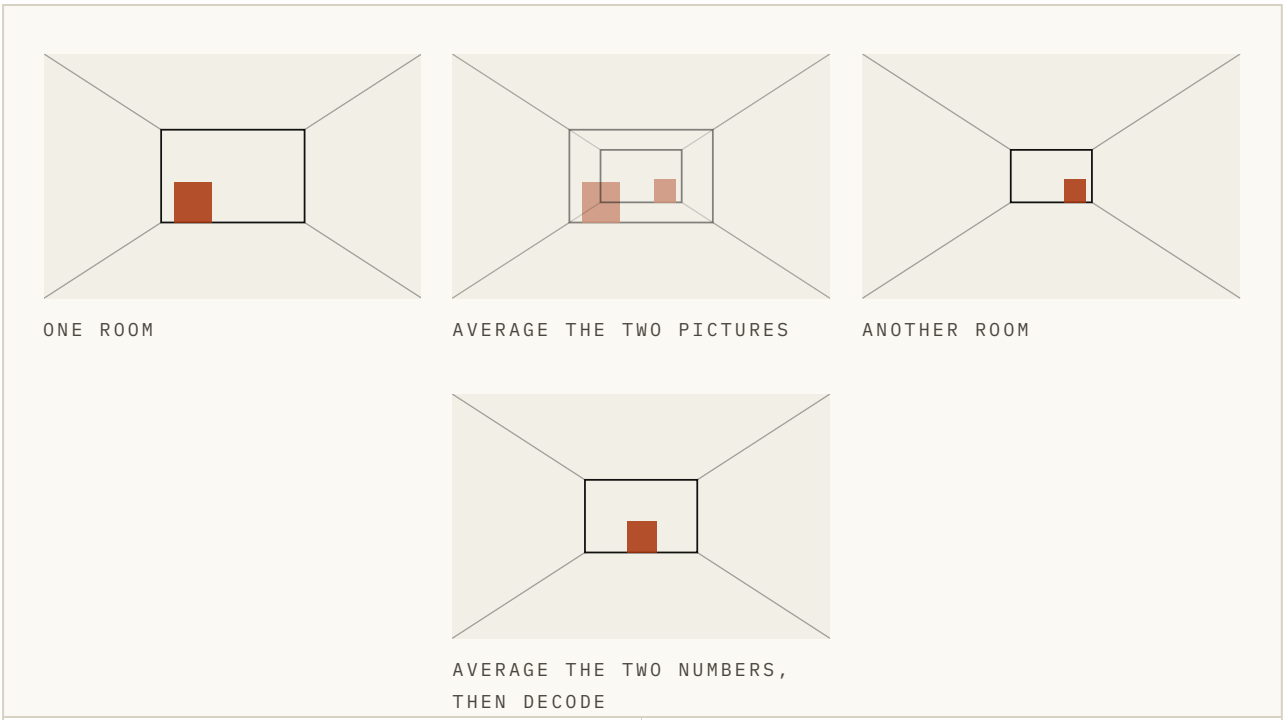


FIG. 4.3 Both rows are honest. Averaging two pictures gives you both rooms at once, faintly, which is not a room and never will be. Averaging the two numbers and decoding gives you a room, because every point in that space is one. Drag the slider and watch which row stays a place you could walk into.

That figure is the reason anyone bothers. Halfway between two pictures is a ghost. Halfway between two descriptions is a room.

A space where the point between two valid things is itself a valid thing is worth a great deal. It is also exactly what pixels will not give you. Prediction, planning and search all involve moving somewhere you have not been. If every step lands on something meaningless, none of them work.

> This is also why the noise in a *variational* autoencoder is not an accident. Deliberately blurring where each picture lands forces neighbouring points to decode to similar things, which is what makes the space navigable rather than a scatter of unrelated addresses.

What makes one good

Small is not the goal. A single number is very small and useless.

The property you want is that the things which matter are easy to get at, and the things that do not are gone. Three tests, roughly in order of how often they are the thing that is actually wrong:

Does it keep what the future turns on? If two situations lead to different outcomes and land on the same point, nothing downstream can tell them apart. That is not recoverable later. Whatever the squeeze threw away is gone.

Do nearby points mean nearby things? A space where a small step can land anywhere is a lookup table with extra steps. Smoothness is what lets you move through it on purpose.

Is it easy to predict forward? A description can be perfectly faithful and still be a nightmare to run forward in time. That sounds like somebody else's problem, and it is not: the point of compressing at all was to get something you could roll forward cheaply.

What it costs you

Rebuilding the picture is a proxy, and proxies drift.

The reconstruction test asks whether the short list can regenerate what the camera saw. What you wanted to know is whether it kept what the decision turns on. Those two come apart in a specific and annoying way. The things that occupy the most pixels are usually not the things that matter. So a description scored on rebuilding pixels spends itself on texture and weather, and quietly loses the small fast object you needed to avoid.

There is a second cost, subtler. Anything the squeeze drops is unrecoverable from that point on. Everything after the bottleneck works from the short list and nothing else. So a choice made here, early, by a part of the system that has no idea what task is coming, quietly sets the ceiling on everything after it.

That is a strange place to put a decision that important. It is also why so much of the field's argument is about the middle of the pipe rather than either end.

Try it

1 A camera hands you tens of thousands of numbers per frame. Deciding whether to walk forward needs two or three. What is the bottleneck for?

a. Making the model smaller so it runs faster
b. Forcing everything through a narrow opening, so what survives is what the pictures were actually made of
c. Removing noise from the camera
d. Compressing the file on disk

> ANSWER b. Forcing everything through a narrow opening, so what survives is what the pictures were actually made of. Speed and file size are side effects. The reason to squeeze is that succeeding at the squeeze requires recovering the things that generated the picture in the first place.

2 Nobody hand-picks what the short list should contain. Why does it end up holding the right things anyway?

a. The architecture has one unit per concept
b. If the pictures were generated by a few things changing, recovering those things is the cheapest way to rebuild them
c. The training labels say so
d. The decoder is told the answer

> ANSWER b. If the pictures were generated by a few things changing, recovering those things is the cheapest way to rebuild them. The ordering falls out of the pressure rather than being designed. That is the appeal, and also why the ordering is only roughly the one you wanted.

3 Average two pictures of two different rooms. What do you get?

a. A room halfway between them
b. Both rooms at once, faintly, which is not a room
c. The first room
d. An empty picture

> ANSWER b. Both rooms at once, faintly, which is not a room. This is what pixel-space blending actually does. It is the clearest reason to want a space where the point between two valid things is itself valid.

4 Average the two short descriptions instead, then decode. What do you get?

a. The same ghost
b. A room, because every point in that space decodes to one
c. Nothing, the numbers do not add
d. A picture of both rooms side by side

> ANSWER b. A room, because every point in that space decodes to one. Prediction, planning and search all involve moving somewhere you have not been. That only works if the places in between mean something.

5 Why is the noise in a variational autoencoder not just a nuisance?

a. It makes training faster
b. It hides the training data
c. Blurring where each picture lands forces neighbouring points to decode to similar things
d. It reduces the number of parameters

> ANSWER c. Blurring where each picture lands forces neighbouring points to decode to similar things. Smoothness is the property that makes the space navigable. Without it you have a lookup table with extra steps.

6 Which of these is NOT what makes a compact description a good one?

a. It keeps what the future turns on
b. Nearby points mean nearby things
c. It is as small as possible
d. It is easy to step forward in time

> ANSWER c. It is as small as possible. A single number is very small and useless. Small is a consequence of keeping the right things, never the goal on its own.

7 A description is scored on how well it can rebuild the camera image. What can go wrong?

a. Nothing, that is exactly the right test
b. It spends itself on whatever occupies the most pixels, which is usually not what the decision turns on
c. It becomes too small to be useful
d. It stops being differentiable

> ANSWER b. It spends itself on whatever occupies the most pixels, which is usually not what the decision turns on. Reconstruction is a proxy. Texture and weather are most of the pixels; the small fast object you needed to avoid is not.

8 Why is the width of the bottleneck an uncomfortable place to put an important decision?

a. It is hard to tune
b. Everything after it works only from the short list, so whatever was dropped is gone, and it was dropped before anyone knew the task
c. It uses too much memory
d. It has to be chosen before training

> ANSWER b. Everything after it works only from the short list, so whatever was dropped is gone, and it was dropped before anyone knew the task. The loss is not recoverable downstream. A part of the system with no idea what is coming quietly sets the ceiling on everything after it.

FIG. 4.4 Eight questions on the squeeze and the space it produces. The last two are about the parts that are easy to agree with and hard to keep hold of.

Everything here has been one moment at a time. A picture goes in, a short description comes out, and nothing has moved yet.

Thanks for reading. Chapter 5 is about what happens when you start stepping that description forward. The machinery that turns a state and an action into the next state, and the error that creeps in every time you ask it to.

Sources

Auto-Encoding Variational Bayes KINGMA & WELLING, 2013 ↗

The variational autoencoder. The noise it adds is the reason the space ends up navigable instead of a scatter of unrelated addresses.

https://arxiv.org/abs/1312.6114

Reducing the Dimensionality of Data with Neural Networks

HINTON & SALAKHUTDINOV, 2006 ↗

The bottleneck argument before it had modern machinery behind it. Paywalled.

https://doi.org/10.1126/science.1127647

World Models HA & SCHMIDHUBER, 2018 ↗

Every frame crushed to thirty-two numbers, and everything after that working only from those. The clearest example of the squeeze in a working agent.

https://arxiv.org/abs/1803.10122

Learning Latent Dynamics for Planning from Pixels (PlaNet) HAFNER ET AL., 2018 ↗

Encodes pixels to a compact state and then plans in it, without ever rebuilding a frame in order to decide anything.

https://arxiv.org/abs/1811.04551

Neural Discrete Representation Learning (VQ-VAE) VAN DEN OORD ET AL., 2017 ↗

What happens when the short list is forced to be a handful of discrete symbols rather than continuous numbers.

https://arxiv.org/abs/1711.00937

beta-VAE HIGGINS ET AL., 2017 ↗

Turn the pressure on the bottleneck up and the axes start to line up with things you can name. Also a good demonstration of what that costs.

https://openreview.net/forum?id=Sy2fzU9gl

Early Visual Concept Learning with Unsupervised Deep Learning

HIGGINS ET AL., 2016 ↗

The argument that a good representation is one whose directions mean something, written before it was a crowded field.

https://arxiv.org/abs/1606.05579

Representation Learning: A Review and New Perspectives

BENGIO, COURVILLE & VINCENT, 2013 ↗

Still the best single statement of what a learned representation is supposed to be for, and of how many of these questions were already open.

https://arxiv.org/abs/1206.5538

FIG. 4.5 Ordered for someone building on this rather than for historical completeness.